

MINI PROJECT ON
VERISCOPE TRADE

**A Smart Retail Assistant for Sales Tracking and
Product Authentication**

**Submitted in partial fulfillment of the requirement
for the award of the degree of
Bachelor of Computer Applications**



**UTTARANCHAL
UNIVERSITY
DEHRADUN**

**NAAC
GRADE A+**

Uttaranchal university

Uttaranchal school of computing science

Semester-IV

Under the supervision of

Mr.Anup Shekher Chamoli

Assist. Proffesor

USCS

Submitted by:

Kartik Sharma

Enroll:UU2409000159

Sec:E

ACKNOWLEDGEMENT

The successful completion of this mini project would not have been possible without the guidance and support of many individuals. I would like to express my sincere gratitude to all those who directly or indirectly helped me during the development of this project titled **“VeriScope Trade – Smart Retail Assistant for Sales Tracking and Product Authentication.”**

Primarily, I would like to thank **Prof. (Dr.) Sonal Sharma, Director-USCS**, for providing a motivating and supportive academic environment that encouraged learning and innovation.

I am deeply thankful to **Dr. Sameer Dev Sharma, Head of Department-USCS**, for providing the necessary resources and valuable support for the successful completion of this project report.

I express my heartfelt thanks to **Mr. Anup shekhar chamoli, Project Mentor, Assistant Professor-USCS**, for her continuous guidance, encouragement, and valuable suggestions throughout the project development process. Her support and mentorship greatly helped me in completing this project successfully.

Kartik Sharma

Enroll. No:UU2409000159

BCA IV(E)

DECLARATION

I hereby declare that the project report entitled “**VeriScope Trade – Smart Retail Assistant for Sales Tracking and Product Authentication**” submitted by **Kartik Sharma** to Uttaranchal School of Computing Sciences is an original work carried out under the guidance of **Mr. Anup shekher chamoli, Assistant Professor-USCS**.

I further declare that the work presented in this project report has not been submitted, either in part or in full, for the award of any other degree, diploma, or certification in this university or any other university or institute.

Kartik Sharma

Enroll. No:UU2409000159

BCA IV(E)

CERTIFICATE OF ORIGINALITY

This is to certify that **Kartik Sharma**, student of **BCA 4th Semester**, of **Uttaranchal School of Computing Sciences, Dehradun**, has successfully completed the Project Report on the topic “**VeriScope Trade – Smart Retail Assistant for Sales Tracking and Product Authentication**” for the session **(2025-26)**.

Under the supervision of

Mr. Anup Shekher Chamoli

Assist. Professor

USCS

TABLE OF CONTENTS

S. No	Topic	Page No.
1	Acknowledgement	1
2	Declaration	2
3	Certificate of Originality	3
4	Introduction	4
5	Objectives of the Project	5
6	System analysis .preliminary investigations .feasibility study .project planning .gantt chart .DFD model .control flow diagram .state diagram .ER model diagram .flow chart	6-16
7	Software Requirement specification	17
8	Software & Hardware Requirements	18
9	Software Engineering Paradigm Applied	19
10	System Design	20-23
11	Future Scope	24
12	testing	25-28
13	conclusion	29
14	Appendices	30-37
15	Bibliography	38-39
16	References	39

INTRODUCTION

Small retail shops and local vendors generate important business data every day in the form of handwritten sales records, expense notes, and product information. However, most small shopkeepers still rely on traditional notebooks instead of digital systems. This makes it difficult to calculate profit or loss, identify fast-selling or slow-moving products, analyze long-term business trends, or verify the authenticity of products being sold. As a result, decision-making becomes slow, inaccurate, and sometimes risky due to the possibility of unknowingly selling counterfeit goods.

To address these real-world challenges, this project presents **Veriscopetrade**, a Python based mobile application designed to digitize handwritten sales data, provide intelligent business insights, and support counterfeit product detection. The system enables users to capture images of their handwritten sales notebooks through a mobile device. Using Optical Character Recognition (OCR) technology, the application converts the extracted text into structured digital records, eliminating the need for manual data entry and making digital adoption simple for non-technical users.

Once digitized, the data is securely stored and analyzed to generate daily, weekly, monthly, and yearly sales reports. The system identifies high-demand and low-demand products, calculates expenses and overall profit or loss, and provides AI-based suggestions to improve stock management and business performance. In addition, the application includes a product authenticity verification feature where shopkeepers can scan batch numbers or barcodes to determine whether a product is genuine or suspicious. Aggregated reports, trends, and heatmaps help highlight high-risk batches or regions associated with counterfeit goods.

The application is designed to function even in low-connectivity environments by supporting offline data storage with automatic synchronization when internet access becomes available. Voice input, manual correction options, and a simple user interface ensure accessibility for users with limited digital literacy.

Overall, Veriscopetrade delivers a practical, scalable, and user-friendly digital solution that empowers small shopkeepers to manage sales efficiently, gain meaningful business insights, prevent counterfeit risks, and make informed, data-driven decisions for sustainable growth.

OBJECTIVE

The primary objective of this project is to develop a smart mobile-based business management system for small shopkeepers and local vendors who rely on handwritten records and lack proper tools for sales analysis, profit calculation, and product authenticity verification. Manual record-keeping often leads to errors, poor decision-making, and difficulty in tracking business performance.

This project aims to create a user-friendly application that converts handwritten sales data into digital records, securely stores and analyzes the information, and generates meaningful sales insights and reports. It also includes a feature to verify product authenticity through batch number or barcode scanning to help prevent counterfeit product sales.

By automating these processes, the system seeks to reduce manual effort, improve data accuracy, support better stock and profit management, enable offline usability, and promote data-driven decision-making for sustainable small-business growth.

SYSTEM ANALYSIS

Identification of Need

Small shopkeepers and local vendors often manage their daily sales, expenses, and product details using handwritten notebooks instead of digital systems. This traditional method makes it difficult to calculate profit or loss, track product demand, analyze long-term business trends, and maintain accurate records. Manual calculations are time-consuming and prone to human error, which can negatively affect business decisions and overall growth.

Another major concern is the lack of a reliable mechanism to verify product authenticity. Shopkeepers may unknowingly sell counterfeit or suspicious products because they do not have access to verification tools or centralized information about product batches. This can lead to financial loss and reduced customer trust.

Therefore, there is a strong need for a simple, affordable, and easy-to-use digital solution that can convert handwritten records into digital data, provide meaningful business insights, and help detect counterfeit products. The proposed system fulfills this need by offering automated record digitization, sales analysis, authenticity verification, offline usability, and secure data management within a mobile application designed specifically for small retail businesses.

Preliminary Investigation

The preliminary investigation focused on understanding the current working methods of small shopkeepers and identifying the key challenges they face in daily business management. It was observed that most small vendors rely on handwritten notebooks to record sales, expenses, and stock details. This manual approach makes data management slow, error-prone, and difficult to analyze for profit calculation or business planning.

Discussions and observations also revealed the absence of affordable digital tools that are simple enough for non-technical users. Existing business management applications are often complex, require continuous internet access, or do not support handwritten data conversion. Additionally, shopkeepers lack reliable systems to verify product authenticity, increasing the risk of counterfeit sales.

Based on this investigation, it became clear that a mobile-based solution with OCR-enabled digitization, offline functionality, simple analytics, and product verification features would effectively address these real-world problems. These findings formed the foundation for designing the proposed mechanism and architecture of the project.

Feasibility Study

The feasibility study evaluates whether the proposed system can be successfully developed and implemented in real-world conditions for small shopkeepers and local vendors.

- **Technical Feasibility:** The application can be developed using Java-based mobile technologies along with OCR libraries for handwritten text recognition and secure cloud or local storage for data management. These technologies are widely available, cost-effective, and suitable for mobile devices, making technical implementation achievable.
- **Economic Feasibility:** The system is designed to be affordable for small business owners. It does not require expensive hardware or complex infrastructure, as it can operate on standard smartphones with optional internet connectivity. This makes the solution financially practical for local vendors.
- **Operational Feasibility:** The application provides a simple user interface, voice input support, offline functionality, and easy navigation, ensuring that even users with limited digital knowledge can operate it comfortably. This increases the likelihood of real-world adoption and effective daily use.
- **Schedule Feasibility:** The project can be completed within the planned academic development timeline using available tools, predefined modules, and structured implementation phases.

Overall, the feasibility study confirms that the proposed system is practical, affordable, user-friendly, and achievable within the given technical and time constraints, making it suitable for small-scale retail environments.

Project Planning

Project planning for the proposed system focuses on organizing development activities, defining timelines, and ensuring systematic completion of each module within the academic schedule. The planning process includes requirement analysis, system design, implementation, testing, and final deployment of the mobile application.

Initially, user requirements and real-world problems faced by small shopkeepers were studied to define system features such as OCR-based record digitization, sales analysis, product authenticity verification, offline data handling, and secure storage. After requirement analysis, the system architecture, database structure, and user interface design were planned to ensure simplicity and usability.

The development phase is divided into manageable stages, including mobile application coding, OCR integration, analytics module creation, and testing of verification features. Regular testing and validation are planned to identify errors early and improve system reliability. Finally, documentation and user guidance are prepared to support smooth usage and project submission.

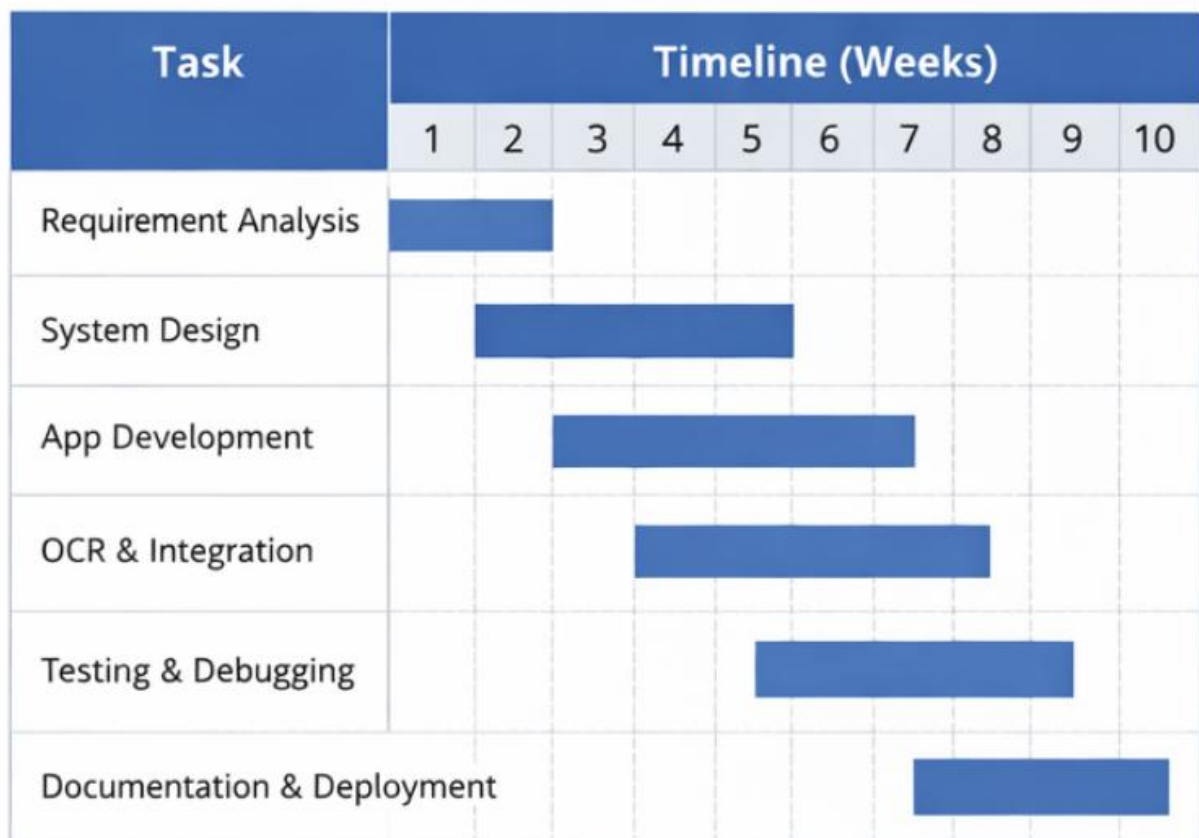
This structured project planning ensures timely completion, efficient resource utilization, and successful delivery of a functional and user-friendly business management application for small retail vendors.

Gantt Chart

The Gantt chart represents the project timeline in a time-based visual schedule. The estimated duration of each phase is:

- Requirement Analysis – 1 week
- System Design – 1 week
- Development – 3 weeks
- Integration – 2 weeks
- Testing – 2 weeks
- Documentation & Deployment – 1 week

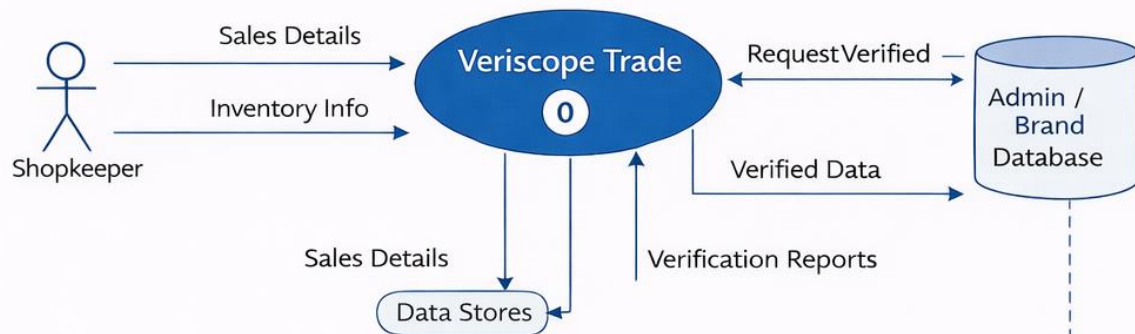
The total estimated project duration is **10 weeks**, with some overlapping of development, integration, and testing phases to optimize time utilization and ensure smooth progress.



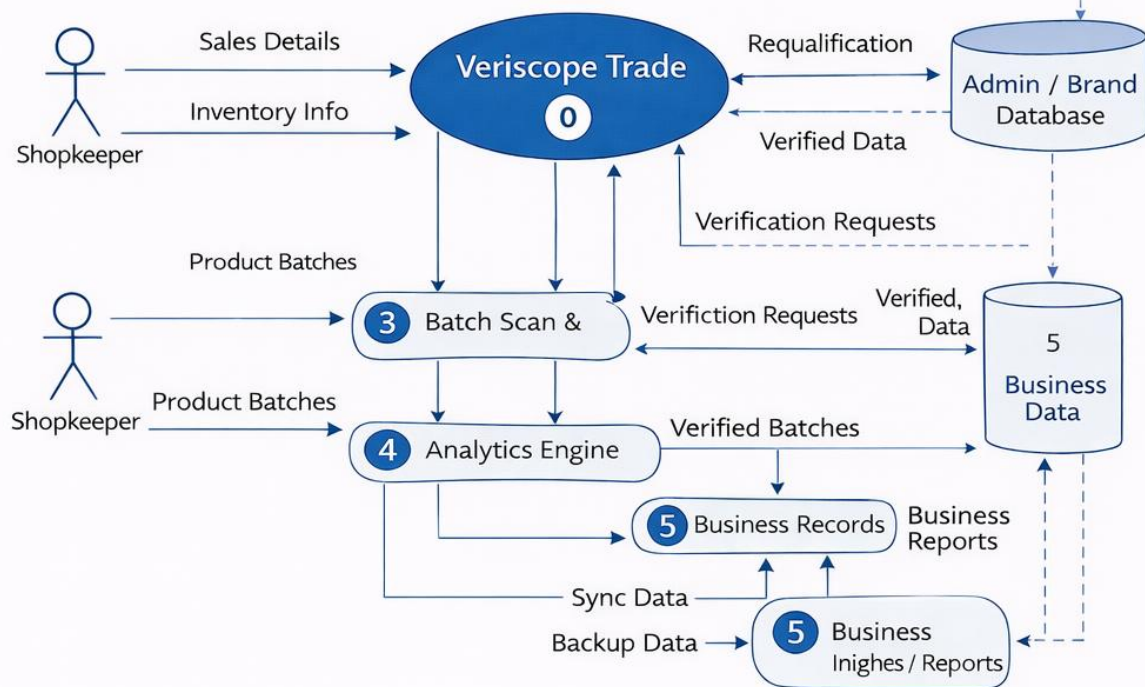
System Model

Data Model (DFD – Data Flow Diagram)

Veriscope Trade - DFD Level 1



Veriscope Trade - DFD Level 1



● Process



External Entity / Actor

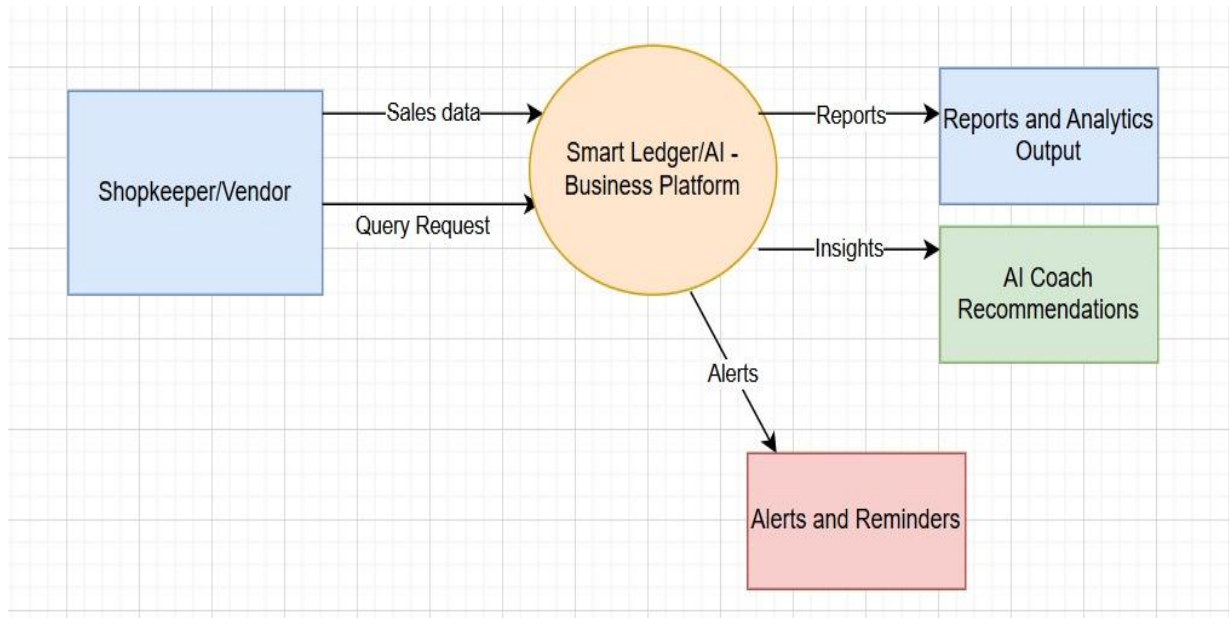
→ Data Flow



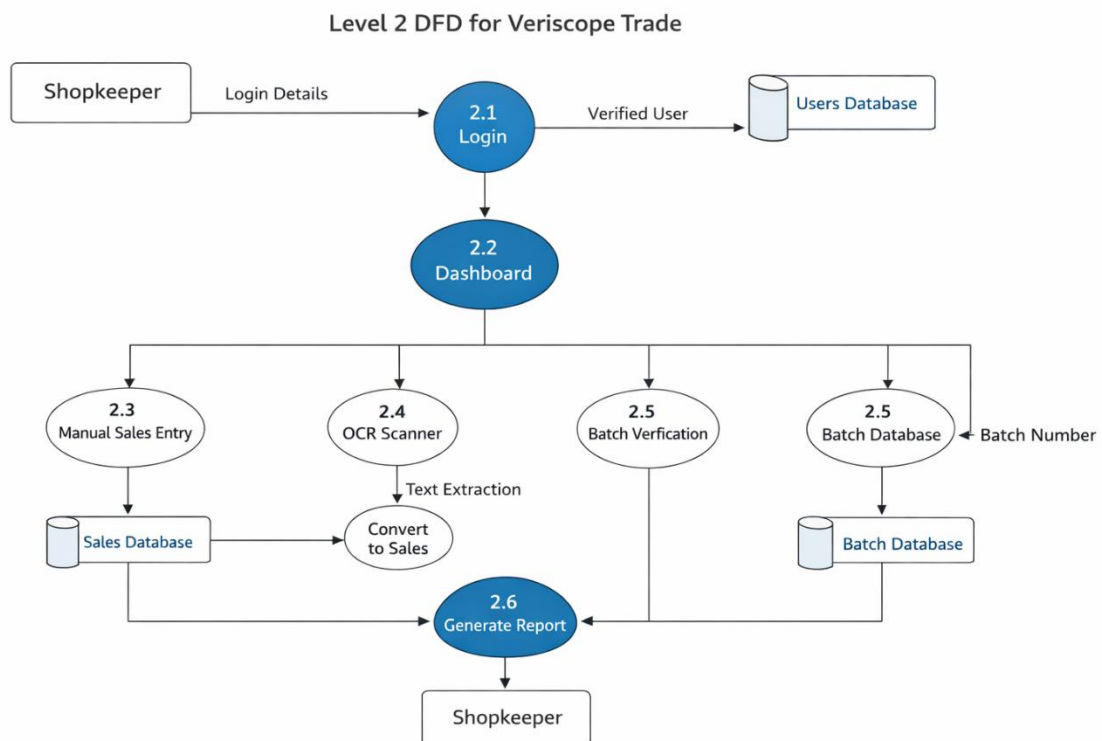
Data Store

○ Start

DFD Level – 0:

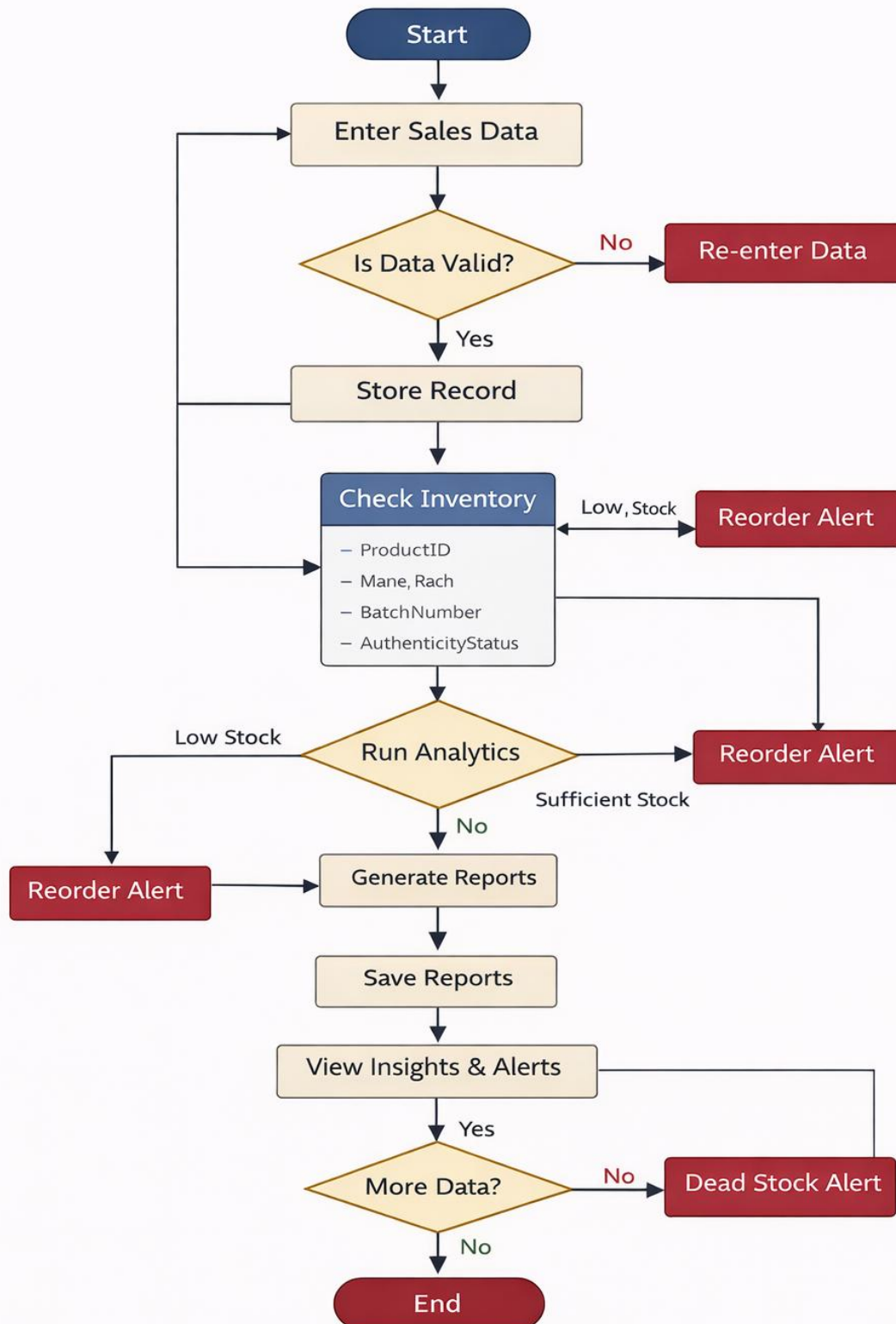


LVL-2 DFD:



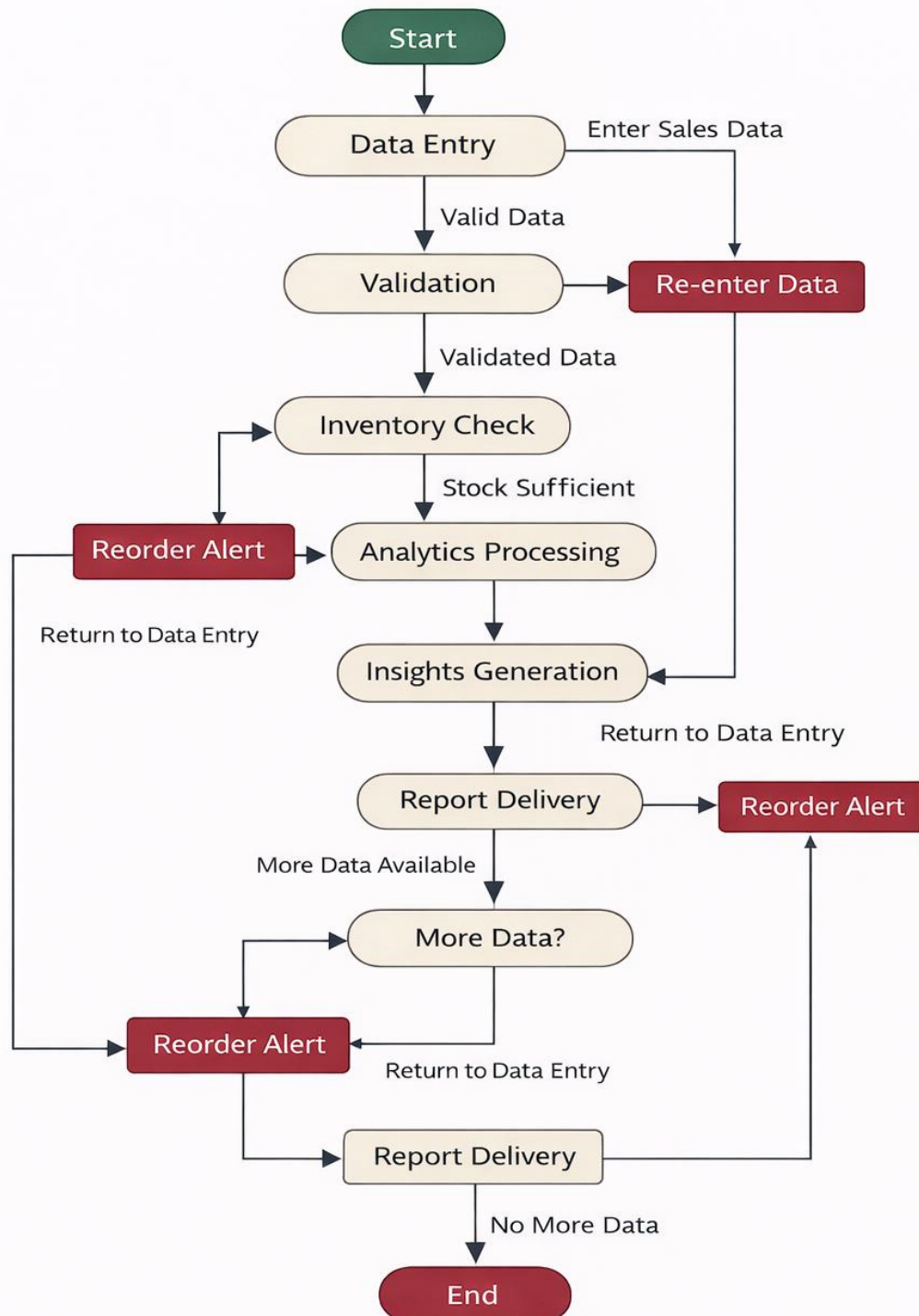
Control Flow Diagram

CONTROL FLOW DIAGRAM

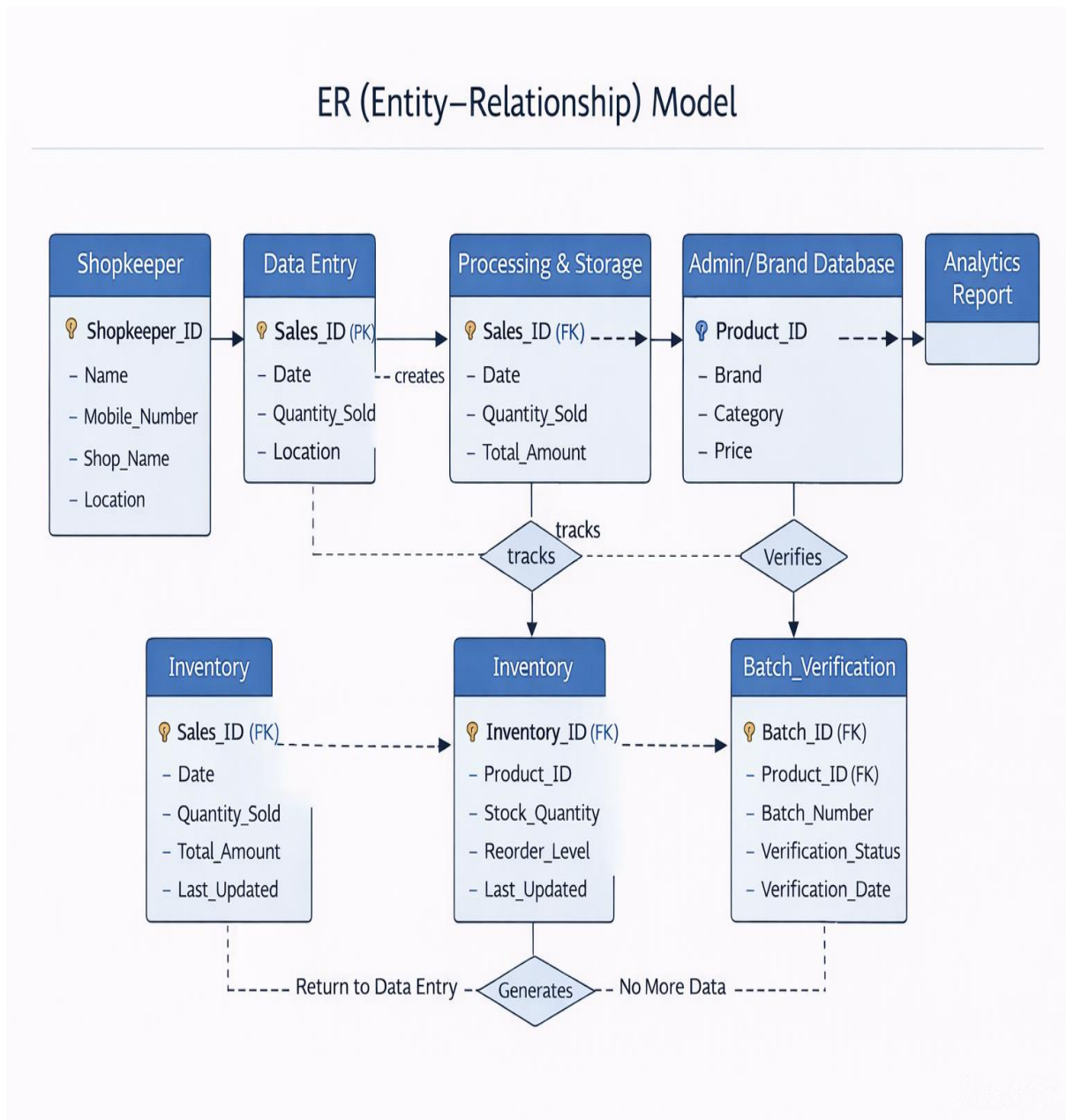


State Diagram

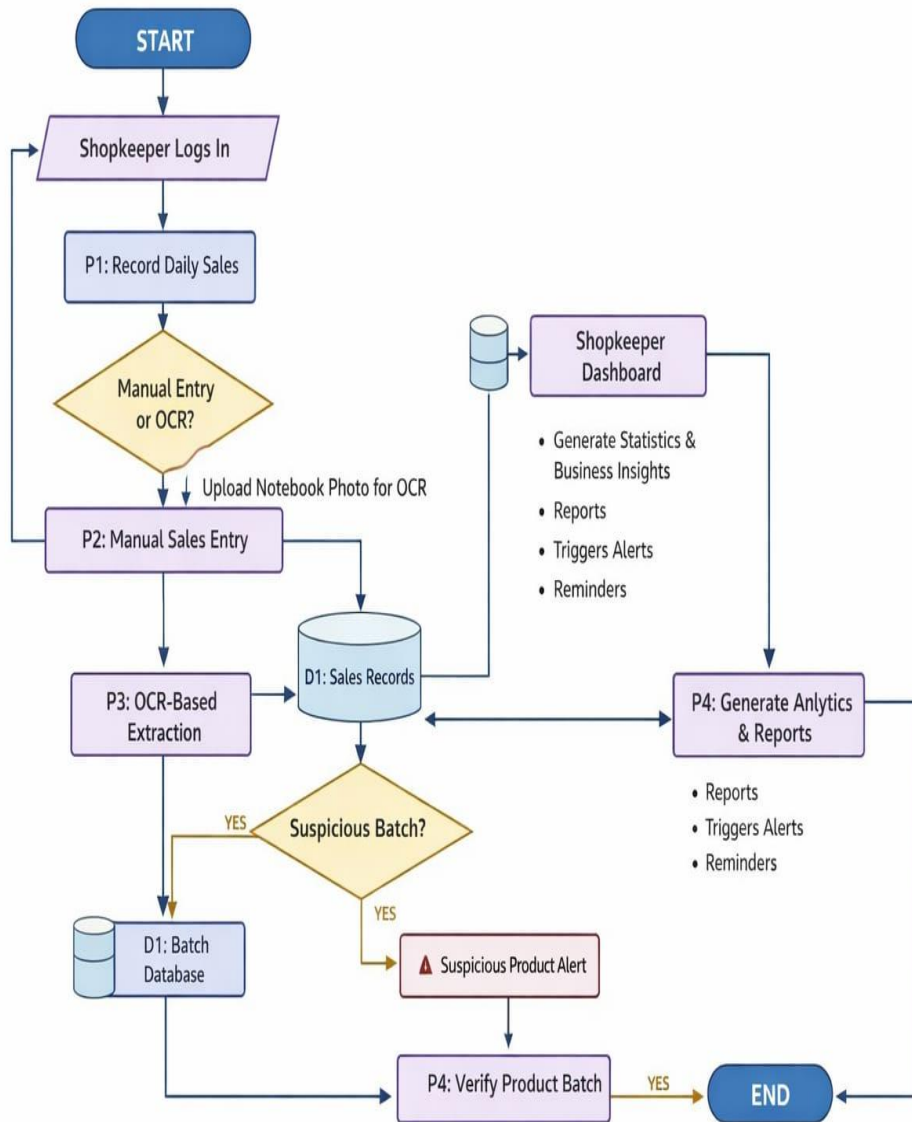
STATE DIAGRAM



Entity–Relationship (ER) Model diagram



FLOWCHAR



SOFTWARE REQUIREMENT SPECIFICATIONS (SRS)

The Software Requirement Specification defines the functional and non-functional requirements of the proposed mobile application to ensure proper system development, performance, and usability for small shopkeepers and local vendors. The system is designed to digitize handwritten sales records, provide meaningful business insights, and support product authenticity verification in a simple and secure manner.

Functional Requirements

The application allows users to capture images of handwritten sales records and convert them into structured digital data using OCR technology. It securely stores and manages sales, expense, and product information while enabling easy retrieval when required. The system generates daily, weekly, monthly, and yearly sales reports along with automatic profit or loss calculations. It also provides useful business insights such as identification of fast-moving and slow-moving products, stock reminders, and simple improvement suggestions. In addition, the application supports barcode or batch number scanning to verify product authenticity and detect suspicious or counterfeit items. Offline data entry is supported, and the system automatically synchronizes stored data when an internet connection becomes available.

Non-Functional Requirements

The application is designed with a simple and user-friendly interface suitable for individuals with limited digital knowledge. It ensures data security, privacy, and reliable storage with backup and recovery capabilities. The system is expected to provide fast response time, accurate OCR recognition, and stable performance on standard smartphones. Furthermore, the design allows scalability to handle increasing business data and flexibility for future feature enhancements.

The hardware requirements for operating the system are minimal and include a smartphone with camera support, sufficient storage capacity, and basic internet connectivity for synchronization. On the software side, the application is developed using Java-based mobile development technologies, integrated OCR libraries for text recognition, and secure local or cloud-based storage mechanisms for data management and analytics.

Hardware Requirements

A smartphone with camera support, minimum required storage, and basic internet connectivity for synchronization is sufficient to operate the system.

Software Requirements

The system is developed using Python-based development tools, OCR libraries for text recognition, and secure storage or cloud synchronization technologies for data management and analytics.

SOFTWARE ENGINEERING PARADIGM APPLIED

The development of the **Veriscope Trade** system follows the **Incremental Software Development Model**, which is a widely used software engineering paradigm where the system is designed, developed, and tested in small functional modules or increments. Instead of building the entire system at once, the project is developed step by step, with each increment adding new features and improvements to the system.

This approach is particularly suitable for the Veriscope Trade project because the system consists of multiple independent modules such as user authentication, shop registration, sales management, OCR-based data extraction, batch verification, and analytics. By developing the system incrementally, each module can be designed, implemented, and tested individually before integrating it with the complete system.

In the first increment, the basic system functionality is developed, including user registration, login authentication, and shop information setup. In the next increment, the sales entry module and dashboard are implemented to allow shopkeepers to record and view their daily sales data. Subsequent increments include advanced features such as OCR-based notebook scanning for automatic sales data extraction, batch number verification for identifying suspicious products, and reminder notifications for missing sales entries.

The Incremental Model provides several advantages for this project. It allows early testing of system components, reduces development risk, and enables gradual improvements based on system performance. Additionally, it makes the system easier to maintain and extend in the future, as new features such as inventory management, advanced analytics, or mobile integration can be added without redesigning the entire system.

Therefore, the Incremental Software Development Paradigm is used in this project to ensure systematic development, efficient testing, and continuous enhancement of the Veriscope Trade system.

SYSTEM DESIGN

System design is an important phase in software development that focuses on defining the overall architecture, components, data flow, and interactions within the system. The Veriscope Trade system is designed using a modular, scalable, and user-friendly approach to ensure efficiency, accuracy, and reliability for small shopkeepers.

1. Modularisation Details

Modularisation refers to dividing a system into smaller, independent, and manageable modules so that each module performs a specific function. The Veriscope Trade application is divided into the following major modules:

a) Data Entry Module

This module allows shopkeepers to digitize handwritten sales records using Optical Character Recognition (OCR). It captures sales data, validates it, and converts it into digital format for storage and analysis.

b) Inventory Management Module

This module manages stock levels, tracks product availability, identifies slow-moving items, and generates reorder alerts when stock reaches a minimum threshold.

c) Batch Scan and Verification Module

This module scans product QR codes or batch numbers to verify authenticity. It communicates with the brand database to detect counterfeit products.

d) Analytics Engine Module

This module processes sales and inventory data to generate business insights such as profit/loss analysis, trend forecasting, and performance reports.

e) Reporting and Insights Module

This module provides graphical and tabular reports to shopkeepers for better decision-making.

f) Synchronization Module

This module ensures secure data backup and synchronization between local storage and cloud servers.

Each module works independently while maintaining proper communication with other modules, ensuring system reliability and scalability.

2. Data Integrity and Constraints

Data integrity ensures accuracy, consistency, and reliability of data stored in the system. The Veriscope Trade system implements the following constraints:

a) Entity Integrity

Each table has a unique primary key such as Product ID, Sales ID, or User ID to ensure that every record is uniquely identifiable.

b) Referential Integrity

Foreign keys are used to maintain relationships between tables. For example, Sales records are linked to Product records through Product ID.

c) Domain Constraints

Data types and validation rules are applied to ensure correct data entry. For instance:

- Quantity must be a positive integer.
- Price must be a numeric value.
- Dates must follow a standard format.

d) Validation Rules

Input validation ensures that incomplete or incorrect data is not stored in the system.

3. Database Design

The Veriscope Trade system uses a relational database model to store and manage data efficiently.

Main Database Tables

a) User Table

Stores shopkeeper login details and profile information.

- User_ID (Primary Key)
- Name
- Phone Number
- Password

b) Product Table

Contains product details and stock information.

- Product_ID (Primary Key)
- Product Name

- Batch Number
- Price
- Quantity

c) Sales Table

Stores digitized sales transaction records.

- Sales_ID (Primary Key)
- Product_ID (Foreign Key)
- Quantity Sold
- Date

d) Verification Table

Stores product authenticity verification results.

- Verification_ID (Primary Key)
- Product_ID
- Batch Status
- Verification Date

e) Reports Table

Stores generated business insights and analytics results.

The database is designed using normalization techniques to avoid data redundancy and improve efficiency.

4. User Interface Design

The user interface of Veriscope Trade is designed to be simple, intuitive, and user-friendly, especially for small shopkeepers who may have limited technical knowledge.

Design Principles

a) Simplicity

The interface uses clear icons, minimal text, and easy navigation to ensure usability.

b) Consistency

Uniform layout, color schemes, and menu structures are maintained throughout the application.

c) Accessibility

Large buttons, readable fonts, and simple instructions make the system accessible to all users.

Main UI Screens

1. **Login Screen** – Secure user authentication.
2. **Dashboard Screen** – Displays key business statistics and quick actions.
3. **Sales Digitization Screen** – Allows scanning of handwritten records.
4. **Inventory Screen** – Shows stock details and alerts.
5. **Verification Screen** – Used for batch scanning and authenticity checking.
6. **Reports Screen** – Displays charts, trends, and business insights.

FUTURE SCOPE AND FURTHER ENHANCEMENTS

The Veriscope Trade system has significant potential for future development.

1. AI-Based Demand Prediction

Future versions can use machine learning to predict sales demand.

2. Mobile Payment Integration

Integration with digital payment systems can be added.

3. Multi-User Support

Multiple shop staff can access the system simultaneously.

4. Real-Time Cloud Analytics

Advanced cloud-based analytics can improve decision-making.

5. Voice Input Support

Voice recognition can help less literate users.

6. Barcode Auto-Billing System

Automatic billing features can be integrated.

Testing

1. Unit Testing

Objective: Test individual functions or modules separately to ensure they work correctly.

Tested Components:

- addProduct()
- authenticateProduct()
- recordSale()
- showSalesReport()

Purpose:

- Ensure each function performs correctly with valid input.
 - Verify accurate product authentication and sales calculations.
-

2. Integration Testing

Objective: Verify communication between different modules of the system.

Tested Components:

- Product Module + Sales Module
- Sales Module + Report Module

Purpose:

- Ensure data flows correctly between modules.
 - Verify sales are properly connected with product records.
-

3. System Testing

Objective: Test the complete VeriScope Trade system as a whole.

Tested Components:

- Full application workflow
- Authentication process
- Sales tracking system

Purpose:

- Ensure the complete application works without errors.
 - Verify all modules function together smoothly.
-

4. Functional Testing

Objective: Check whether all project functionalities work according to requirements.

Tested Components:

- Product Authentication
- Sales Entry
- Revenue Calculation
- Sales Report Generation

Purpose:

- Ensure every feature gives correct results.
 - Verify business logic of the application.
-

5. Input Testing

Objective: Check system behavior for different user inputs.

Tested Components:

- Valid Product IDs
- Invalid Product IDs
- Quantity Inputs

Purpose:

- Ensure the system handles incorrect inputs properly.
 - Prevent invalid data processing.
-

6. Output Testing

Objective: Verify correctness of outputs generated by the system.

Tested Components:

- Product Details Display
- Authentication Results

- Sales Reports

Purpose:

- Ensure outputs are accurate and well formatted.
 - Verify correct revenue calculations.
-

7. Performance Testing

Objective: Measure system speed and efficiency during operations.

Tested Components:

- Multiple Sales Entries
- Report Generation Speed

Purpose:

- Ensure the system responds quickly.
 - Verify smooth performance under workload.
-

8. Security Testing

Objective: Ensure safety and protection of project data.

Tested Components:

- Product Records
- Sales Data
- Invalid Access Handling

Purpose:

- Prevent unauthorized operations.
 - Protect stored information from errors.
-

9. User Acceptance Testing

Objective: Verify that the system satisfies user requirements.

Tested Components:

- User Interface
- Project Features
- Overall Usability

Purpose:

- Ensure the project is user-friendly.
 - Confirm user satisfaction with the system.
-

10. Regression Testing

Objective: Ensure previously working functions continue operating after modifications.

Tested Components:

- Authentication Module
- Sales Tracking Module
- Report Generation Module

Purpose:

- Detect errors after updates.
- Maintain stability of existing features.

CONCLUSION

The “VeriScope Trade – Smart Retail Assistant for Sales Tracking and Product Authentication” project was successfully developed using Java programming language. The system provides an efficient solution for managing product authentication, recording sales, and generating sales reports in a retail environment.

This project helped in understanding important concepts such as object-oriented programming, data handling, sales management, and system testing. The application successfully verifies authentic products and maintains accurate sales records, which improves reliability and management efficiency.

The project is user-friendly, simple to operate, and capable of reducing manual effort in retail management. All modules were tested successfully and produced accurate results during execution.

Overall, the project achieved its objectives and demonstrated how technology can be used to improve retail sales tracking and product authentication systems effectively.

APPENDICES

CODING

```
import java.util.*;
```

```
class Product {
```

```
    String productId;
```

```
    String productName;
```

```
    double price;
```

```
    boolean authenticated;
```

```
    Product(String productId, String productName,
```

```
            double price, boolean authenticated) {
```

```
        this.productId = productId;
```

```
        this.productName = productName;
```

```
        this.price = price;
```

```
        this.authenticated = authenticated;
```

```
    }
```

```
    void displayProduct() {
```

```
        System.out.println("ID: " + productId);
```

```
        System.out.println("Name: " + productName);
```

```
        System.out.println("Price: Rs." + price);
```

```
    if(authenticated) {  
        System.out.println("Authenticated: Yes");  
    } else {  
        System.out.println("Authenticated: No");  
    }  
}  
}
```

```
class Sale {  
  
    String saleId;  
    Product product;  
    int quantity;  
    double totalAmount;  
  
    Sale(String saleId, Product product, int quantity) {  
  
        this.saleId = saleId;  
        this.product = product;  
        this.quantity = quantity;  
        this.totalAmount = product.price * quantity;  
    }  
  
    void displaySale() {  
  
        System.out.println("Sale ID: " + saleId);  
        System.out.println("Product: " + product.productName);  
    }  
}
```

```
        System.out.println("Quantity: " + quantity);  
        System.out.println("Total Amount: Rs." + totalAmount);  
    }  
}
```

```
public class Main {
```

```
    static HashMap<String, Product> productDatabase =  
        new HashMap<String, Product>();
```

```
    static ArrayList<Sale> salesList =  
        new ArrayList<Sale>();
```

```
    public static void addProduct(String id,  
                                   String name,  
                                   double price,  
                                   boolean authenticated) {
```

```
        Product product =  
            new Product(id, name, price, authenticated);
```

```
        productDatabase.put(id, product);
```

```
        System.out.println("Product Added Successfully!");  
    }
```

```
    public static void authenticateProduct(String productId) {
```

```
Product product = productDatabase.get(productId);

if(product != null) {

    System.out.println("\nProduct Authentication Result:");
    product.displayProduct();

} else {

    System.out.println("Product not found!");
}
}

public static void recordSale(String saleId,
                               String productId,
                               int quantity) {

    Product product = productDatabase.get(productId);

    if(product != null) {

        Sale sale =
            new Sale(saleId, product, quantity);

        salesList.add(sale);
```

```
        System.out.println("Sale Recorded Successfully!");

    } else {

        System.out.println("Invalid Product ID!");
    }
}

public static void showSalesReport() {

    System.out.println("\n===== SALES REPORT =====");

    double totalRevenue = 0;

    for(Sale sale : salesList) {

        sale.displaySale();
        System.out.println("-----");

        totalRevenue =
            totalRevenue + sale.totalAmount;
    }

    System.out.println("Total Revenue: Rs." + totalRevenue);
}

public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);

addProduct("P101", "Smart Watch", 4999, true);
addProduct("P102", "Bluetooth Speaker", 2999, true);
addProduct("P103", "Fake Headphones", 999, false);

while(true) {

    System.out.println("\n===== VeriScope Trade =====");

    System.out.println("1. Authenticate Product");
    System.out.println("2. Record Sale");
    System.out.println("3. View Sales Report");
    System.out.println("4. Exit");

    System.out.print("Enter choice: ");

    int choice = sc.nextInt();

    switch(choice) {

        case 1:

            System.out.print("Enter Product ID: ");

            String authId = sc.next();
```

```
authenticateProduct(authId);
```

```
break;
```

```
case 2:
```

```
System.out.print("Enter Sale ID: ");
```

```
String saleId = sc.next();
```

```
System.out.print("Enter Product ID: ");
```

```
String productId = sc.next();
```

```
System.out.print("Enter Quantity: ");
```

```
int qty = sc.nextInt();
```

```
recordSale(saleId, productId, qty);
```

```
break;
```

```
case 3:
```

```
showSalesReport();
```

```
break;
```

```
case 4:
```

```
System.out.println("Exiting Program...");
```

```
return;
```

```
default:
```

```
System.out.println("Invalid Choice!");
```

```
}
```

```
}
```

```
}
```

```
}
```

BIBLIOGRAPHY

1. Pressman, Roger S.

Software Engineering: A Practitioner's Approach, 7th Edition, McGraw-Hill Education.

(Used for understanding software engineering concepts and system development models.)

2. Ian Sommerville

Software Engineering, 10th Edition, Pearson Education.

(Reference for system design, software requirements specification, and development paradigms.)

3. Flask Official Documentation

Available at: <https://flask.palletsprojects.com>

(Used for learning Flask framework and backend development.)

4. Python Software Foundation

Python Documentation.

Available at: <https://docs.python.org>

(Used for Python programming concepts and libraries.)

5. Tesseract OCR Documentation

Available at: <https://github.com/tesseract-ocr/tesseract>

(Reference for implementing OCR-based text extraction from images.)

6. Mozilla Developer Network (MDN)

Web Development Documentation.

Available at: <https://developer.mozilla.org>

(Used for HTML, CSS, and JavaScript development.)

7. W3Schools Online Web Tutorials

Available at: <https://www.w3schools.com>

(Used for learning frontend web development concepts.)

8. SQLite Documentation

Available at: <https://www.sqlite.org/docs.html>

(Reference for database design and query implementation.)

References:

- [Oracle Java Documentation](#)

Used for understanding Java programming concepts and syntax.

- [W3Schools Java Tutorial](#)

Referred for basic Java programming examples and learning resources.

- [Programiz Java Programming](#)

Used for studying object-oriented programming concepts in Java.